Unit-II

Regular Expressions

Regular Expression : (RE)

Regular expression can be defined as simple algebric expressions that are used to represent set of strings.

Definition of Regular Expression :

Regular expression is generally defined as :

- 'ϵ' is regular expression corresponding to $L = \{\epsilon\}$
- 'ϕ' is " " " to $L = \{\}$
- 'R' is " " " to $L = \{R\}$
- If R_1 & R_2 are two regular expressions then union of those '$R_1 + R_2$' is also regular expression
- If R_1 & R_2 are regular expressions then concatenation of those '$R_1 . R_2$' is also regular expression.
- If 'R' is regular expression then 'R^*' is also regular expression
- Repeating above steps over the given alphabet (Σ) recursively is also a regular expression.

- Regular Expressions are set of characters that represents search patterns

- Upto now we constructed language (L) for the finite automata based on "conditions on strings". But we construct languge based on the "algebric conditions".

Regular Language:

- Regular language can be defined as "the language that satisfies the regular expression is called as "Regular language".

- Regular language can be defined by the finite automata such as:

1. DFA
2. NFA
3. NFA with ϵ
4. Regular Expression.

Examples on Regular expressions:

Example: write the regular expression for the language accepts all combinations of a's ove $\Sigma = \{a\}$

$\Sigma = \{a\}$

$L = \{\epsilon, a, aa, aaa \cdots\}$

Regular Expression (R.E) $= a^*$

* Write regular expression for language that accepts all possible combinations of a's & b's over $\Sigma = \{a, b\}$

given $\Sigma = \{a, b\}$

$L = \{ \epsilon, a, b, aa, bb, ab, aab, abb \cdots \}$

$R.E = (a+b)^*$

* Design R.E for language ends with '00' over $\Sigma = \{0, 1\}$

given $\Sigma = \{0, 1\}$

$L = \{\epsilon, 00, 100, 000, 1100 \cdots \}$

$R.E = (0+1)^* 00$

* Design R.E that starts with '0' and ends with '1' over $\Sigma = \{0, 1\}$

given $\Sigma = \{0, 1\}$

$L = \{01, 001, 011, 0101, 0001 \cdots \}$

$R.E = 0.(0+1)^*.1$

* Any no. of a's followed by any no. of b's and c's $\Sigma = \{a, b, c\}$

given $\Sigma = \{a, b, c\}$

$L = \{\epsilon, abc, aabc, aabbc, abcc, \cdots \}$

$R.E = a^*.b^*.c^*$

* Atleast 1 a followed by 1 b followed by 1 c $\Sigma = \{a, b, c\}$ (4)

given $\Sigma = \{a, b, c\}$

$L = \{ abc, aabc, aabbc, \ldots \}$

Here 'ϵ' is not possible as condition is atleast one

$R.E = a^+ . b^+ . c^+$

* Begins or ends with '00' & '11'

$(00+11)(0+1)^* + (0+1)^*(00+11)$

Identity rules (or) Algebric laws :

- These identity rules or algebric laws are used for ~~simplifying~~ simplifying the regular expressions

- where ϕ - Null set

ϵ - Null string

- If P, Q, R are regular expressions then the rules or laws are as follows:

1. $\phi + R = R$
2. $\phi \cdot R = \phi$
3. $\epsilon \cdot R = R \cdot \epsilon = R$
4. $R + R = R$
5. $R^* \cdot R^* = R^*$
6. $R^* \cdot R = R \cdot R^* = R^*$
7. $\epsilon + R^* \cdot R = \epsilon + R \cdot R^* = R^*$
8. $(R^*)^* = R^*$
9. $(PQ)^* P = P(QP)^*$
10. $(P^* + Q^*)^* = (P^* \cdot Q^*)^* = (P+Q)^*$

Equivalance of two Regular Expressions

(8)

Simplification of Regular Expression

1. $(1+00^*1) + (1+00^*1)(0+10^*1)^*(0+10^*1) = 0^*1(0+10^*1)^*$

sol: $= (1+00^*1)\left(1 + (0+10^*1)^* \cdot (0+10^*1)\right)$

$= (1+00^*1)\left(\epsilon + (0+10^*1)^*(0+10^*1)\right)$

$\left[\because \epsilon \cdot R = R \cdot \epsilon = R \right.$
$\left. \text{i.e } \epsilon = 1 \right]$

$= (1+00^*1)\ (\epsilon + (0+10^*1)^*.(0+10^*1))$

$= (1+00^*1)\ (0+10^*1)^*$

where $[\epsilon + R.R^* = R^*]$

$= 1(\epsilon+00^*)\ (0+10^*1)^*$

$= 1(0^*)\ .\ (0+10^*1)^*$

where $[\epsilon+R.R^* = R^*]$

$= 0^*1(0+10^*1)^*$

Hence proved.

2. $\epsilon + 1^*(011)^*\ (1^*(011)^*)^* = (1+011)^*$

Sol: $= \epsilon + 1^*(011)^*\ (1^*(011)^*)^*$

$= (1^*(011)^*)^*$

where $[\epsilon + R.R^* = R^*]$

$= (1+011)^*$

where $[(P^*.Q^*)^* = (P+Q)^*]$

Hence proved..

Equivalance of Finite automata & Regular Expression :

1. Finite automata to Regular Expression Conversion
2. Regular Expression to Finite auto conversion.

Finite Automata to Regular Expression :

- This Conversion can be obtained by two different methods :
  - i) Arden's theorem
  - ii) State elimination method.

Arden's Theorem :

statement : If 'P' and 'Q' are two regular Expressions and 'P' does not have any 'ϵ' then the equation :

$R = Q + RP$ will have Unique solution

$R = QP^*$

proof :

$R = Q + RP$

Apply $R = QP^*$ in above, then

$= Q + (QP^*) \cdot P$

$= Q + Q \cdot P \cdot P^*$

$= Q(1 + P.P^*)$

$= Q(\epsilon + P.P^*)$ where $\epsilon = 1$

$\boxed{R = QP^*}$ where $\epsilon + R.R^* = R^*$

Hence Proved

Proof 2 : $R = Q + RP$

$= Q + (Q + RP)P$ where $R = Q + RP$

$= Q + QP + RP^2$

$= Q + QP + (Q + RP)P^2$

$= Q + QP + QP^2 + RP^3$

$= Q(1 + P + P^2 + \dots)$

$= Q(\epsilon + P + P^2 + \dots)$

$= QP^*$ where $[\Sigma^* = \epsilon \cup \Sigma^1 \cup \Sigma^2 \dots$
$= \epsilon + \Sigma^1 + \Sigma^2]$

$\boxed{R = QP^*}$

Conversion of finite automata to regular Expression using Arden's method:

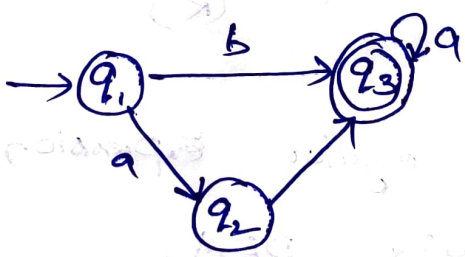
* Using Arden's for conversion we need to follow certain steps. They are:

Step 1: Create equation for each state based on incoming edges.

Step 2: Add 'ϵ' to the initial state equation

Step 3: Simplify the final state equation using Arden's theorem to obtain regular expression.

Example: Convert finite automata to regular Expression



given $\Sigma = \{a, b\}$

Now Create equation for each state

$$q_1 = \epsilon \quad \rightarrow (1)$$

$$q_2 = q_1 a \quad \rightarrow (2)$$

$$q_3 = q_1 b + q_2 a + q_3 a \rightarrow (3)$$

Now Simplify the final state equation q_3

Apply ① & ② in ③

$$q_3 = q_1 b + q_2 a + q_3 a$$

$$= \epsilon . b + q_2 a + q_3 a \quad [\because \text{from } ①]$$

$$= b + q_2 a + q_3 a$$

$$= b + (q_1 a) a + q_3 a \quad [\because \text{from } ②]$$

$$= (b + q_1 aa) + q_3 a$$

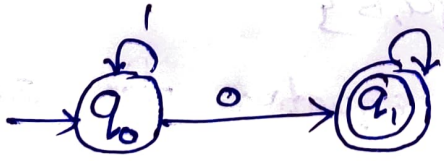
$$= (b + q_1 aa) . a^*$$

$$= (b + \epsilon . aa) . a^* \quad [\because R = Q + RP \text{ to } R = QP^*]$$

$$q_3 = (b + aa) . a^* \quad [\because \text{from } ①]$$

Then Regular expression (R) $= (b + aa) . a^*$

Example: Find the regular expression represented by following DFA.



given $\Sigma = \{0, 1\}$

Now create equation's for all state's.

$$q_0 = q_0 \cdot 1 + \epsilon \rightarrow (1)$$

$$q_1 = q_0 \cdot 0 + q_1 \cdot 0 + q_1 \cdot 1 \rightarrow (2)$$

Now we need to solve final state equation 'q_1'

From (2)

$$q_1 = q_0 \cdot 0 + q_1 \cdot 0 + q_1 \cdot 1$$

From arden's theorem [$\because R = Q + RP$]

$$= q_0 \cdot 0 + q_1 (0+1)$$

$$\boxed{q_1 = (q_0 \cdot 0) \cdot (0+1)^*} \rightarrow (3)$$

[$\because R = QP^*$]

Now we cannot able to simplify, so find the 'q_0' value as we have equation (1)

From (1)

$$q_0 = q_0 \cdot 1 + \epsilon$$

From arden's theorem [$\because R = Q + RP$]

$$= \epsilon \cdot 1^*$$

$$\boxed{q_0 = 1^*} \rightarrow (4)$$

Now apply (4) in (3) equation

$$q_1 = (q_0 \cdot 0) \cdot (0+1)^*$$

$$= (1^* \cdot 0) \cdot (0+1)^*$$

[$\because q_0 = 1^*$, from 4]

$$\boxed{q_1 = (1^* 0)(0+1)^*}$$

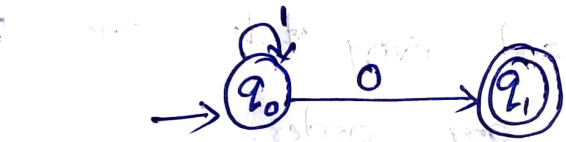
Regular expression (R.E) = $(1^* 0)(0+1)^*$ is obtained.

Conversion of Finite Automata to regular Expression Using "State elimination method":

- To implement state elimination method we need to follow steps:

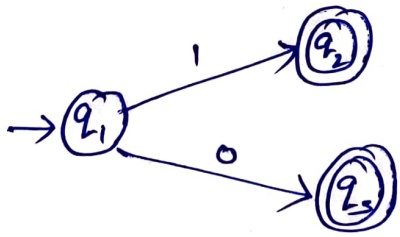
Step 1: Initial state should not have any incoming edge. If exist create new state and make it as initial state.

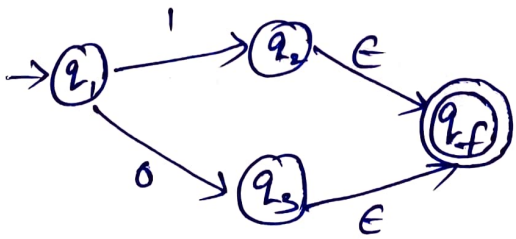
ex:



Step 2: Finite should have single final state. If there exist multiple final states, create new state and make it as final state.

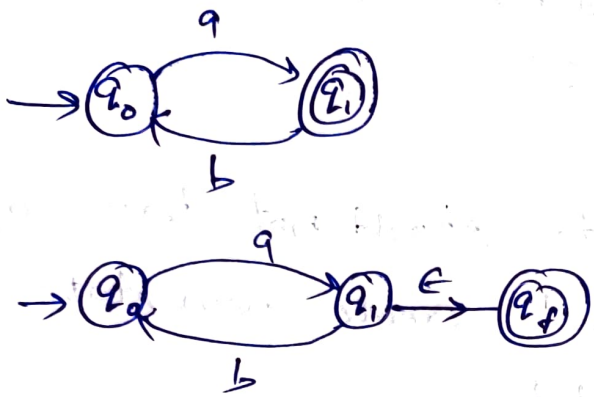
ex:





Step 3: Finale state should not have any out going edge. If Exist create new state and make it as final state.

Ex:



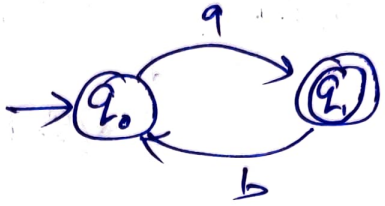
Step 4: Eliminate each and every state one after another without any order.

Step 5: Finite automata should have only two states ?) Initial state

ii) Final state.

Eg: Convert below finite automata into regular expression (F.A) (R.E) using state elimination method.

Note: During conversion we need some "direct method notations". They are

a+b : 

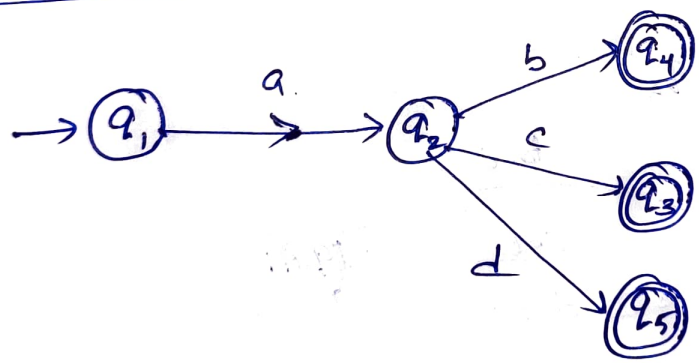
(Union)

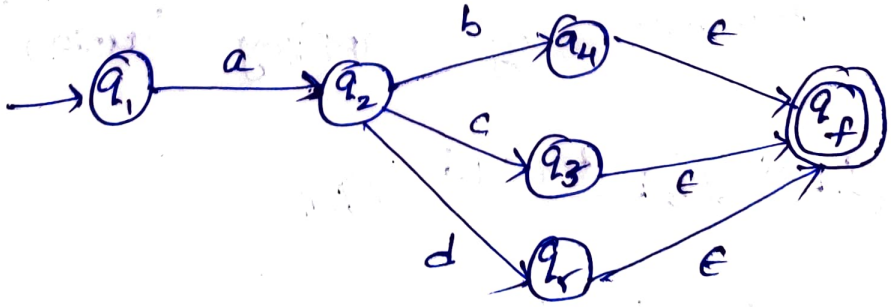
a^* : 

(kleene Closure)

ab : 

(Concatenation)

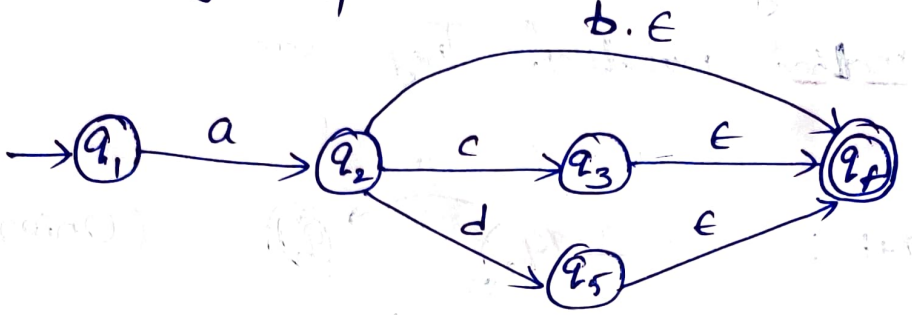


Sol: 

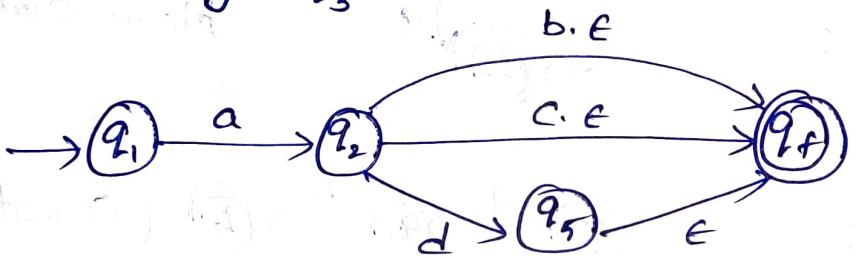
- Here we removed multiple final states by creating new final state 'q_f'

- Now check for different steps to implement, if didn't find any then start eliminating states untill it should remains only two states.

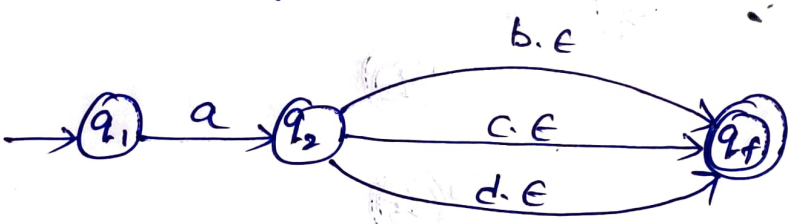
- By eliminating 'q_4'



- By eliminating 'q_3'

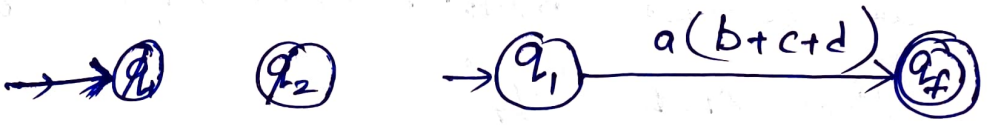


- By eliminating 'q_5'



- Now it becomes by applying 'union'

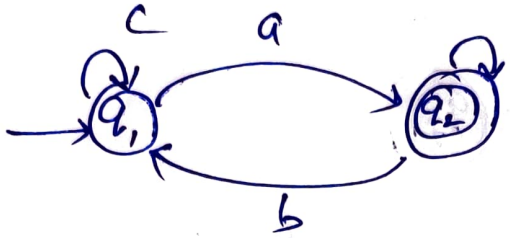
- By eliminating 'q_2' & applying concatenation



- Final Regular expression obtained is between initial state and final state.

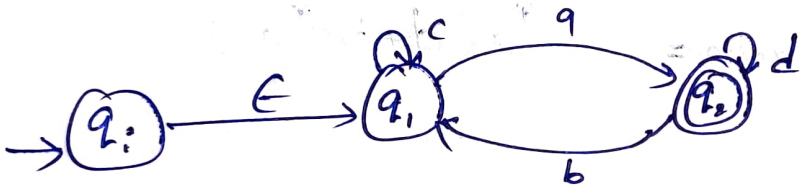
Regular Expression = $a(b+c+d)$

Eg:

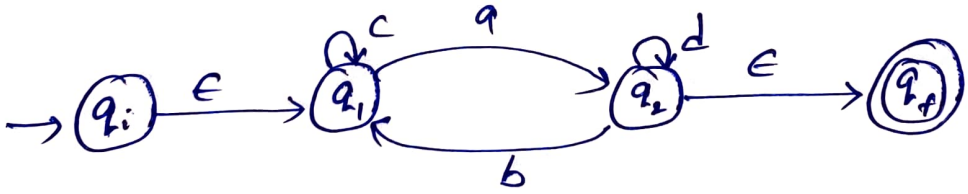


Sol:

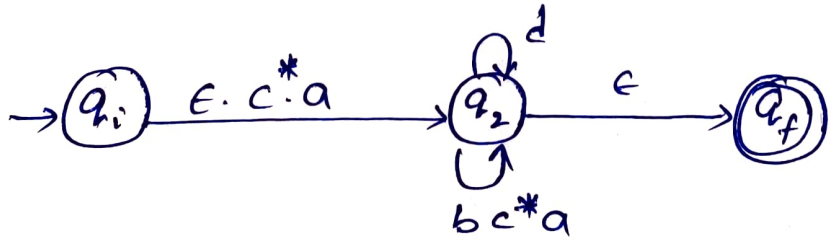
- Initial state 'q_1' has one incoming edge 'c', so add new state and make it as initial state



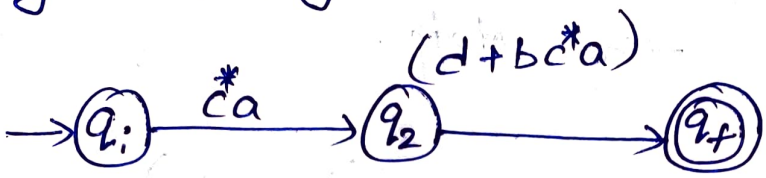
- Final state 'q_2' has one outgoing edge '5', so add new state and make it as final state.



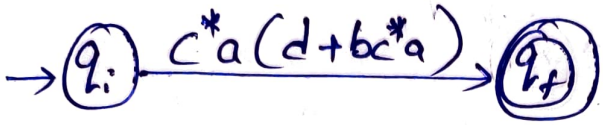
- By eliminating 'q_1'



- By applying union



- By Eliminating 'q_2' & applying Concatenation

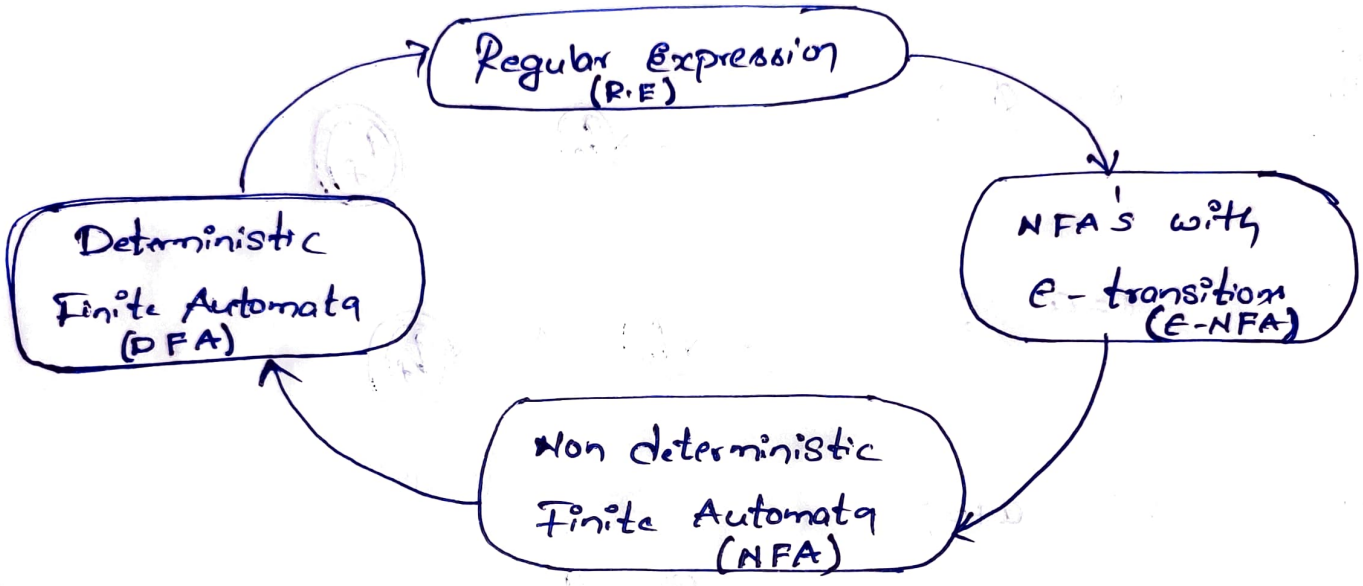


- Final regular Expression obtained in between initial state and final state.

Regular Expression = $c^*a(d+bc^*a)$

Conversion of regular Expression (R.E) to Finite automata (F.A):

- For every regular expression there is an "Equivalent NFA with ε-transition" (i.e ε-NFA).



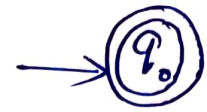
Construction of F.A for a given R.E

- Above diagram also called as relation between R.E, ε-NFA, NFA and DFA
- To construct F.A from given R.E firstly we can directly convert to ε-NFA
- From ε-NFA, we can convert it to the NFA and from NFA we convert it to DFA

Basic Notations for Conversion ; (R.E to F.A)

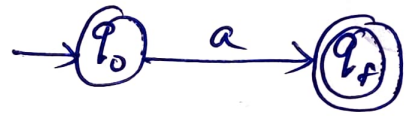
- Let us consider 'r' is regular expression, then the basic notations are as follows:

i. $r = \epsilon$:

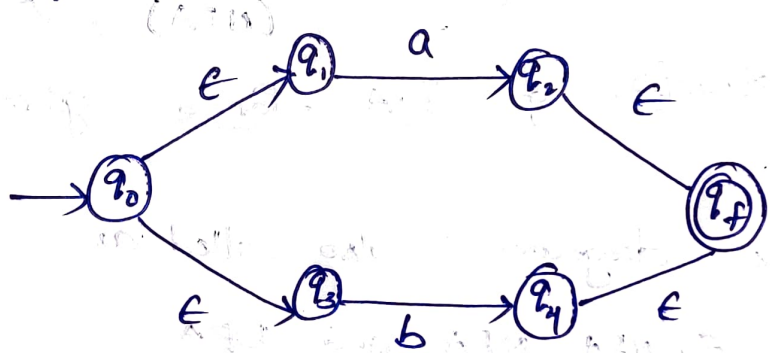


ii. $r = \phi$:

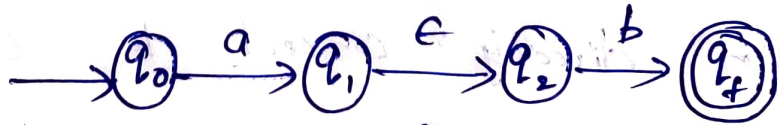
iii. $r = a$:



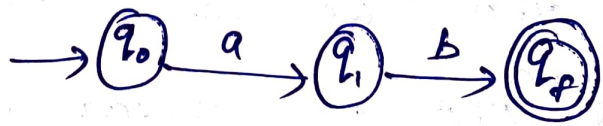
iv. $r = a+b$:



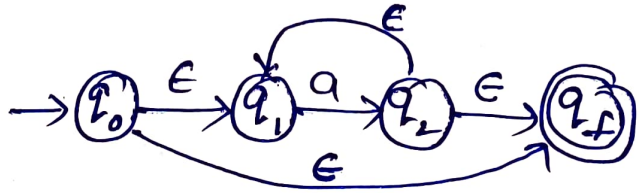
v. $r = a.b$:



(or)



vi. $r = a^*$:

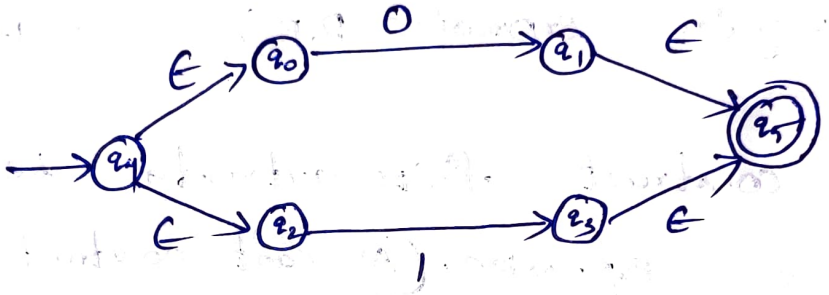


Ex: Convert R.E to F.A of $(0+1)^*.0.1$

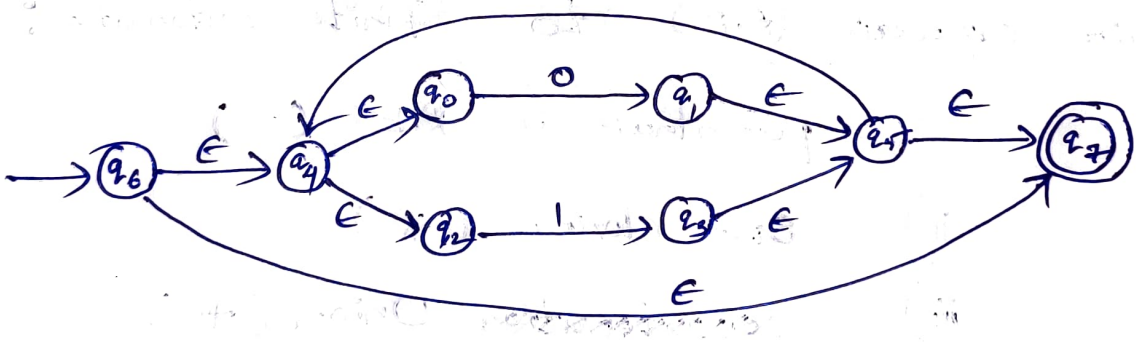
Sol:- Given $(0+1)^*.0.1$

- Let us construct F.A by dividing given R.E into parts.

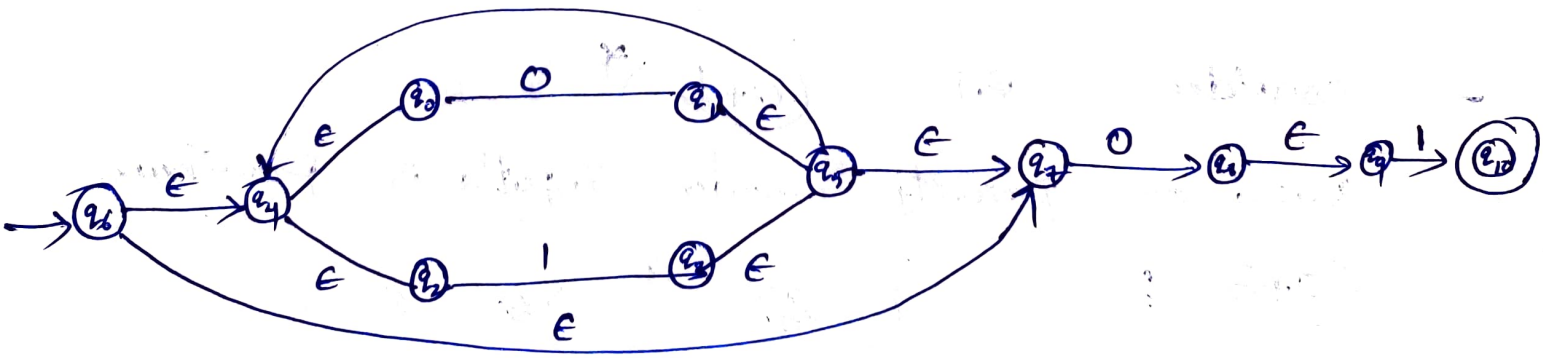
Now consider "0+1"



Now consider $(0+1)^*$



Now consider $(0+1)^*.0.1$



Finally F.A is constructed (ϵ-NFA)

Ex: Construct DFA for Regular language consisting of any no. of a's & b's

Sol:- Let us construct language

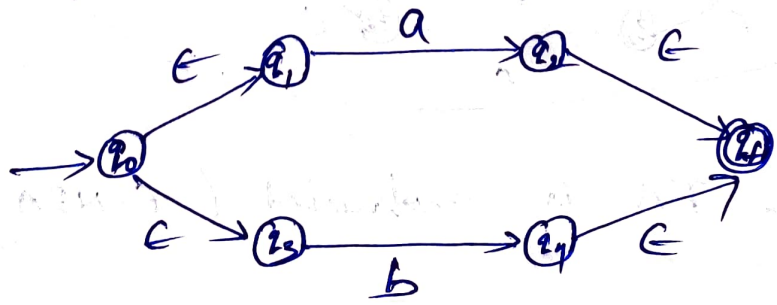
Language (L) = { ab, aabb, abb, aab ... }

Now Regular Expression (R.E) = $(a+b)^*$

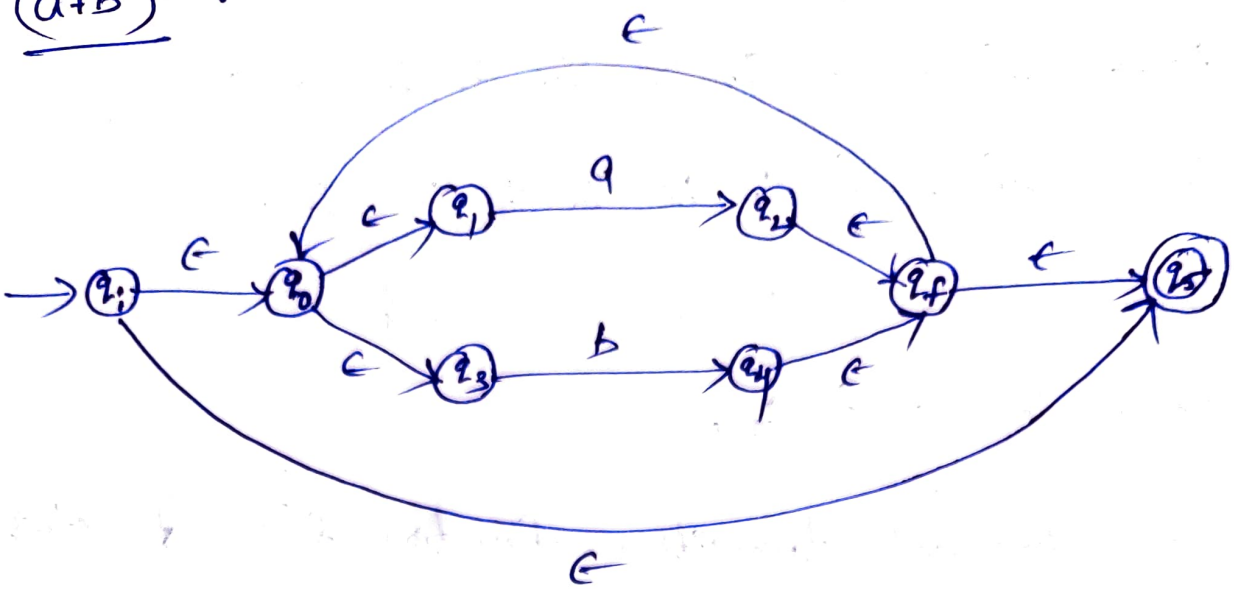
- Let us construct finite automata i.e "ϵ-NFA" for above expression. (As can't construct DFA directly)
- Follows the below priority during conversion of Regular expression (R.E) to Finite automata:
  - i) paranthesis in R.E ( )
  - ii) Star closure (*)
  - iii) ~~Concatenation~~ Union (+)
  - iv) Concatenation (.)
- consider R.E = $(a+b)^*$

Based on priority consider input's in Paranthesis

$(a+b)$:



$(a+b)^*$:



Steps for converting ϵ-NFA to DFA :

Step 1: we will write ϵ-closure of all states starting with initial state.

Step 2: Now find the extended transition function (δ') of all states ϵ-closure results with inputs

Step 3: Repeat step (2) until there is no new result

Step 4: Mark the state of DFA as a final state which contain final state of NFA.

Now follow above steps,

ϵ - closure $(q_i) = \{ q_i, q_0, q_1, q_3, q_5 \}$

ϵ - closure $(q_0) = \{ q_0, q_1, q_3 \}$

ϵ - closure $(q_1) = \{ q_1 \}$

ϵ - closure $(q_3) = \{q_3\}$

ϵ - closure $(q_2) = \{q_2, q_f, q_0, q_1, q_3, q_5\}$

ϵ - closure $(q_4) = \{q_4, q_f, q_0, q_1, q_3, q_5\}$

ϵ - closure $(q_f) = \{q_f, q_0, q_1, q_3, q_5\}$

ϵ - closure $(q_5) = \{q_5\}$

Now find extended transition function (δ') of state starts with initial state.

$$\epsilon\text{ - closure } \{q_i\} = \{q_i, q_0, q_1, q_3, q_5\} = A$$

$$\delta'(A, a) = \epsilon\text{-closure}\{\delta((q_i, q_0, q_1, q_3, q_5), a)\}$$

$$= \epsilon\text{-closure}\{\delta(q_i, a) \cup \delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_3, a) \cup \delta(q_5, a)\}$$

$$= \epsilon\text{- closure } \{q_2\}$$

$$\delta'(A, a) = \{q_2, q_f, q_0, q_1, q_3, q_5\} = B$$

Now, $\delta'(A, b) = \epsilon\text{-closure}\{\delta((q_i, q_0, q_1, q_3, q_5), a)\}$

$$= \epsilon\text{- closure}\{\delta(q_i, b) \cup \delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_3, b) \cup \delta(q_5, b)\}$$

$$= \epsilon\text{- closure } \{q_4\}$$

$$\delta'(A, b) = \{q_4, q_f, q_0, q_1, q_3, q_5\} = C$$

Now $\delta'(B, a) = \epsilon\text{-closure}\{(\delta(q_2, q_f, q_0, q_1, q_3, q_5), a)\}$

$\delta'(B, a) = \epsilon\text{-closure}\{q_2\} = B$

$\delta'(B, b) = \epsilon\text{-closure}\{(\delta(q_2, q_f, q_0, q_1, q_3, q_5), b)\}$

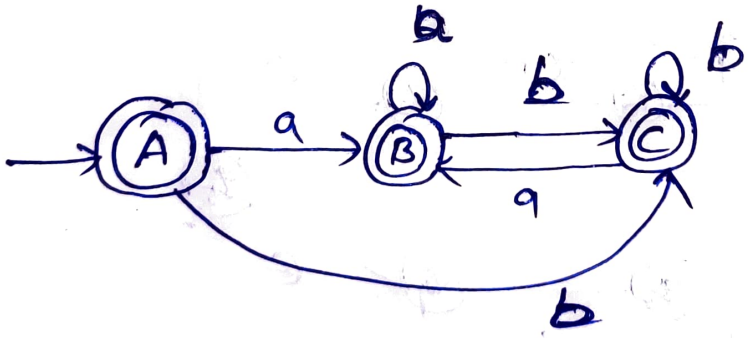
$\delta'(B, b) = \epsilon\text{-closure}\{q_4\} = C$

- Final states are the result that contains 'q_5' in ϵ-closure results, so 'q_5' is in A, B, C
  i.e $F = \{A, B, C\}$

Transition Table :

State	a	b
(A)	B	C
(B)	B	C
(C)	B	C

Transition Diagram :



DFA Obtained

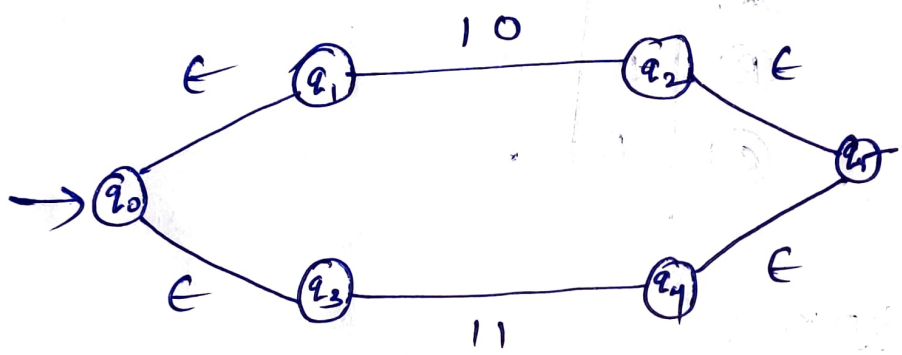
Ex: Construct NFA to the given Regular Expression

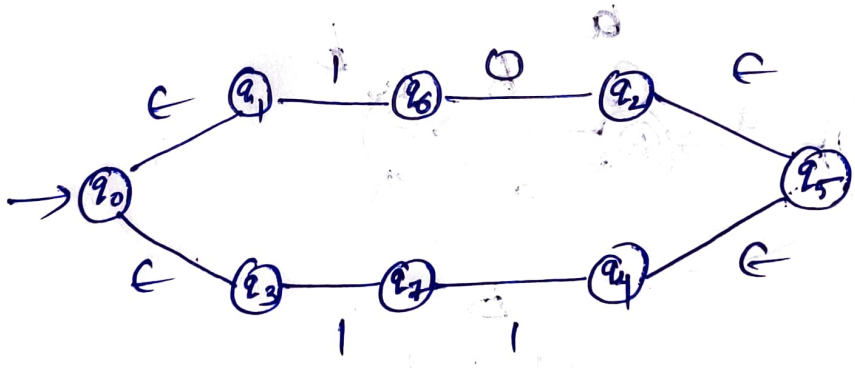
$$R.E = (10+11)^*00$$

Sol: - Let us Construct NFA by R.E Solving step by step

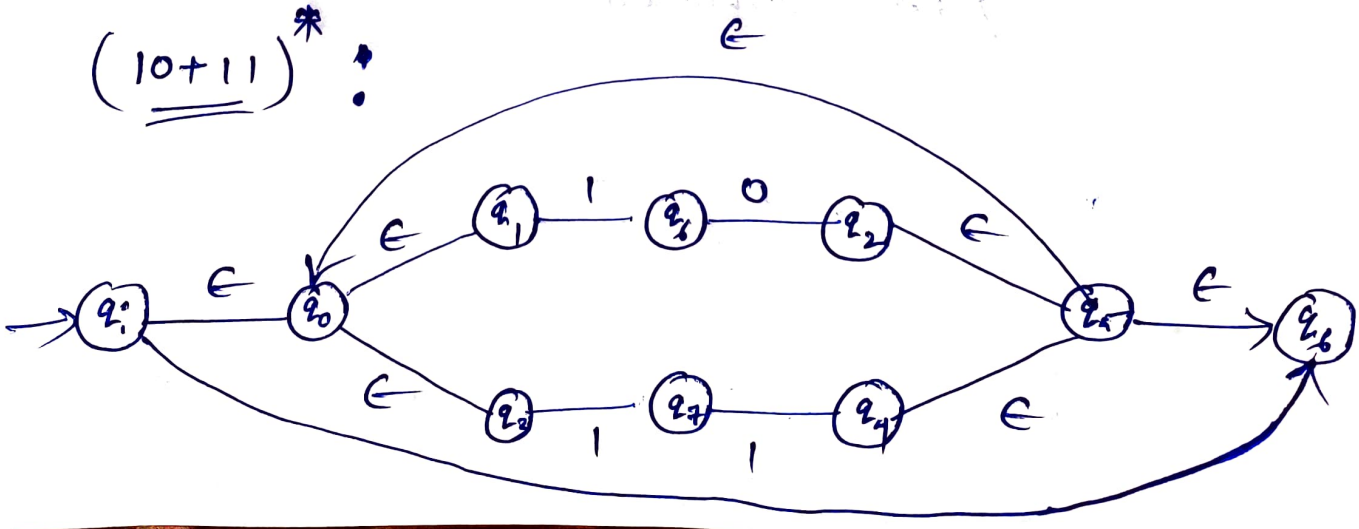
- As ( ) & * are at one place, first priority will be "( )" only.

10+11 ;

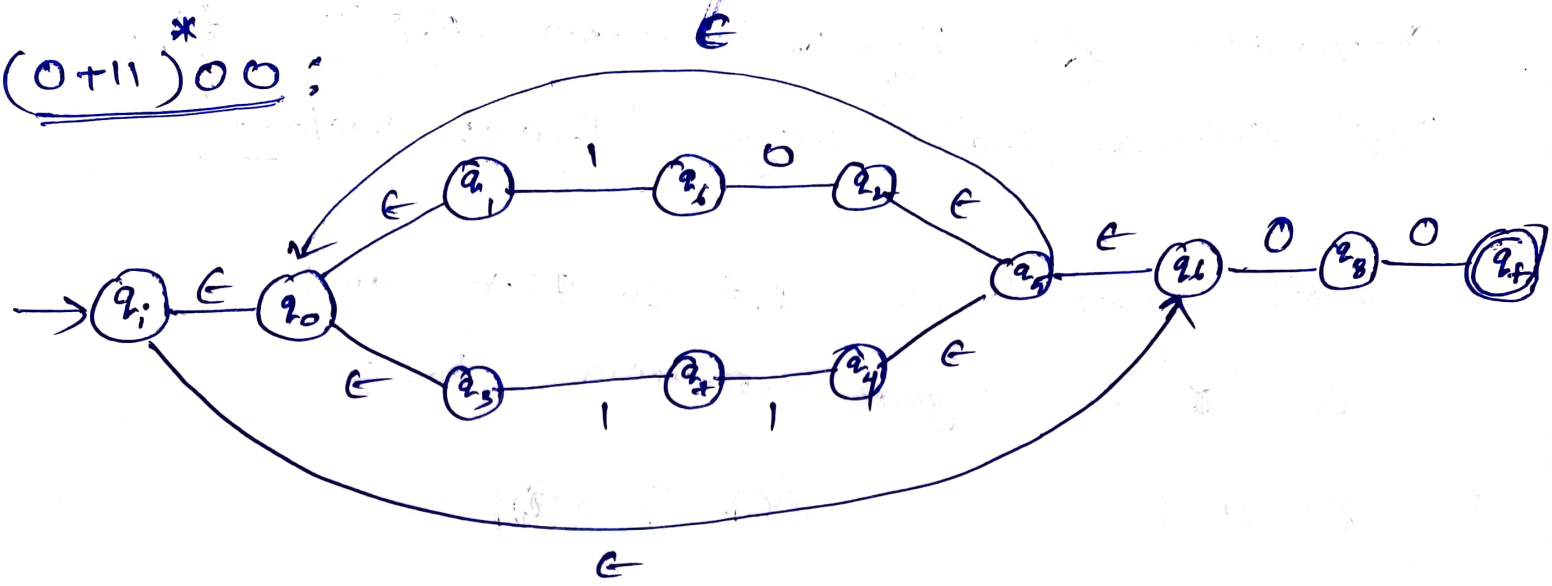




$(10+11)^*$:



$(0+11)^*00$:



ϵ-NFA

- Now we need to convert it to ϵ-NFA

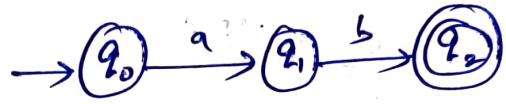
(Refer ϵ-NFA to NFA in Unit-1)

Conversion of Regular Expression (R.E) to Finite Automata (F.A) using "Direct method".

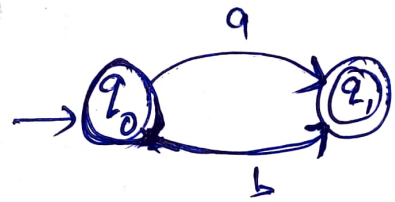
- Basic notations for R.E to F.A are:
- If 'r' is regular expression then:

i. $r = a$:

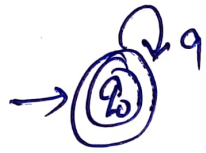
ii. $r = ab$:



iii. $r = a+b$:



iv. $r = a^*$

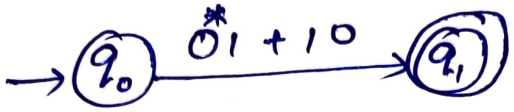


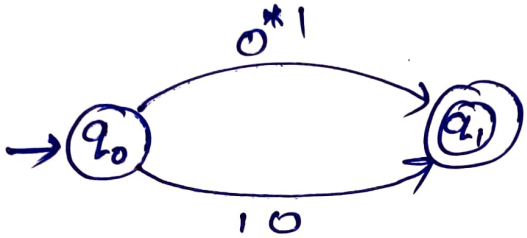
v. $r = (a+b)^*$

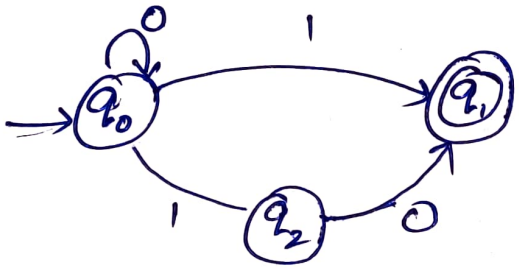
Example: Convert following regular Expressions into finite automata using direct method.

i) $0^*1 + 10$:

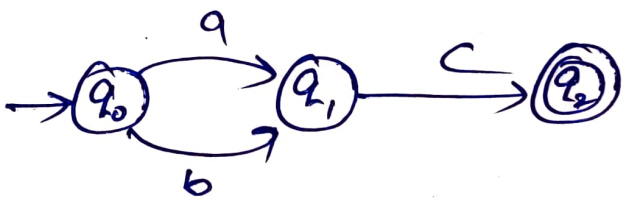
Sol:



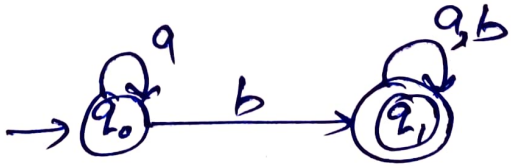




ii) $a^*b(a+b)^*$:



iii) $a^*.b(a+b)^*$:



Construction of DFA for given regular expression: (Using Direct Method)

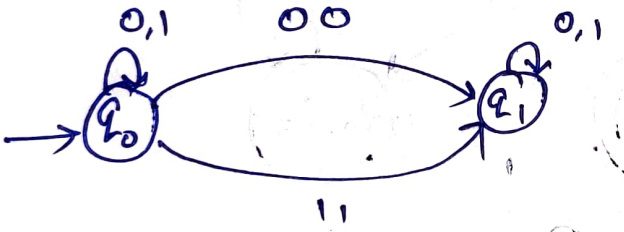
(8)

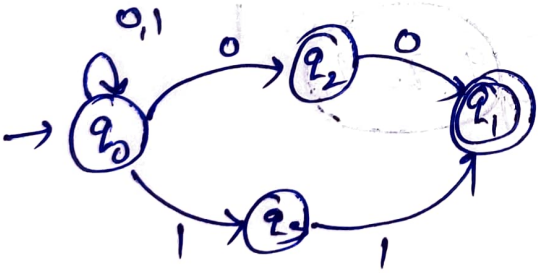
Regular Expression to DFA: (Using Direct method)

Eg: $(0+1)^*(00+11)(0+1)^*$

Sol:- $R.E = (0+1)^*(00+11)(0+1)^*$

By using direct method we can draw transition diagram as follows:





(NFA)

Now draw transition table for above NFA:

δ	0	1
q_0	$\{q_0 q_1\}$	$\{q_0 q_3\}$
q_1	q_2	ϕ
q_2	q_2	q_2
q_3	ϕ	q_2

Now let us try to construct table for DFA. based on NFA table

δ	0	1
q_0	(q_0, q_1)	(q_0, q_3)
$q_0 q_1$	(q_0, q_1, q_2)	(q_0, q_3)
$q_0 q_3$	(q_0, q_1)	(q_0, q_3)
$q_0 q_1 q_2$	(q_0, q_1, q_2)	(q_0, q_3, q_2)
$q_0 q_3 q_2$	(q_0, q_1, q_2)	(q_0, q_3, q_2)

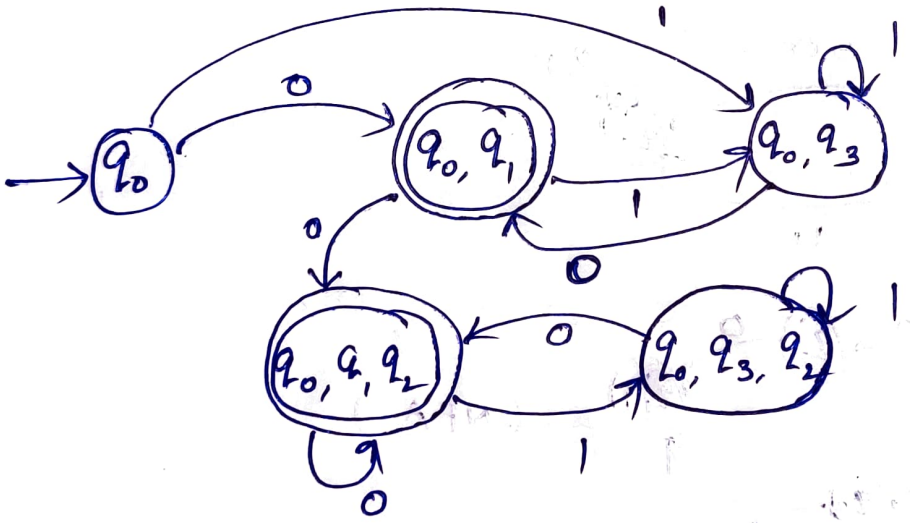
$\delta((q_0, q_1), 0)$

$\Rightarrow \delta(q_0, 0) \cup \delta(q_1, 0)$

$\Rightarrow (q_0, q_1) \cup (q_2)$

$\Rightarrow (q_0, q_1, q_2)$

Now construct DFA,



- Here final states are obtained NFA final state, 'q_1' is final state
- In obtain states where 'q_1' is present, it is final state.

Pumping Lemma : (Substring)

Statement of Pumping lemma :

If substring of a string is repeated many times and if the resultant string is also available in Language (L), then we can say it is result.

Steps to implement Pumping lemma :

1. Consider language as regular
2. Assume a constant 'P' and select the string 'w' from 'L' such that $|w| \geq P$
3. Divide 'W' as xyz (3 substrings)

   i.e $|y| \geq 0$

   $|xy| \leq P$

   for every $i \geq 1$ every string xy^iz belongs to 'L'

- Finally by using pumping lemma we going to prove given language is not regular.

Ex: Prove language $L = \{ a^n b^n \mid n \geq 0 \}$ is not a regular language

Sol: Let us construct language 'L'

$$L = \{ ab, aabb, aaabbb \cdots \}$$

Assume $P = 6$, & $\omega = aaabbb$

Now check $|\omega| \geq P$

$$|aaabbb| \geq P$$

$$6 \geq 6$$

So condition satisfied.

$\therefore |\omega| \geq 6$

Now divide 'ω' into xyz format

$$\underbrace{aa}_{x}\underbrace{ab}_{y}\underbrace{bb}_{z}$$

Now Check $|y| \geq 0$

$2 \geq 0$ (satisfied)

and we check $|xy| \leq P$

$4 \leq 6$ (Satisfied)

Now $xy^i z$ when $i=1$

$xyz \Rightarrow aaabbb \quad (\in L)$

if $xy^i z$ when $i=2$

$xy^2 z \Rightarrow aa(ab)^2 bb$

$\Rightarrow aaababbb \quad (\notin L)$

So it is proved that one of substring not present in Language 'L', so given language is not regular.

Ex: Prove that $L = \{ a^p \mid p \text{ is prime} \}$ is not a regular.

Sol:

Let us construct language 'L'

$$L = \{ aa, aaa, aaaaa \ldots \}$$

Assume $p = 3$, and $\omega = aaa$

Now check $|\omega| \geq p$

$|3| \geq 3$ (satisfied)

Now divide string into xyz

$\underbrace{a}_{x}\ \underbrace{a}_{y}\ \underbrace{a}_{z}$

Now check $|y| \geq 0$

$1 \geq 0$ (satisfied)

and check $|xy| \leq p$

$|aa| \leq p$

$2 \leq 3$ (satisfied)

Now $xy^i z$ where $i = 1$

$xyz \Rightarrow aaa \in L$

xy^2z where $i = 2$

$xy^2z \Rightarrow a(a)^2 a \Rightarrow aaaa \notin L$

As above string 'aaaa' not present in language 'L' then it is not regular.

Ex: Prove Language $(L) = \{ a^{2n} b^{3n} a^n \mid n \geq 0 \}$ is not regular

Sol:- Let us construct language 'L'

where $n = 1 \Rightarrow a^2 b^3 a^1 \Rightarrow aa\,bbb\,a$

$n = 2 \Rightarrow a^4 b^6 a^2 \Rightarrow aaaa\,bbbbbb\,aa$

Now $L = \{ aabbba, aaaa\,bbb\,bbb\,aa \ldots \}$

Assume $p = 4$ and $w = aabbba$

Check $|w| \geq p$

$6 \geq 4$ (satisfied)

and divide string 'w' into xyz

$\underbrace{aa}_{x}\ \underbrace{bb}_{y}\ \underbrace{ba}_{z}$

Now Check $|y| \geq 0$

$2 \geq 0$ (satisfied)

and Check $|xy| \leq p$

$|aabb| \leq p$

$4 \leq 4$ (satisfied)

Now xy^iz where $i = 1$

$xy^1z \Rightarrow aabbba \in L$

where $i = 2$

$xy^2z \Rightarrow aa(bb)^2ba$

$\Rightarrow aabbbbba \notin L$

As above string 'aabbbbba' is not present in 'L'; so given language is not regular.

Closure properties of Regular language (or) Regular Sets :

- consider Integer ($\underline{I}$) = $\{ I_1, I_2 \cdots \}$

when $I_1 + I_2 = I_3 \in \underline{I}$

- Then integer '$\underline{I}$' is said to be closed under addition (or) Union.

ex: $5 + 3 = 8$

where 5, 3, 8 all belongs to integer group.

- Now apply same concept on regular language

Union ($L_1 \cup L_2$) :

- Here language L_1 & L_2 are closed under union because the result will also be present in regular language.

$L_1 \cup L_2 = L_3 \in$ (Regular language)

Concatenation ($L_1 L_2$) :

- Language L_1 & L_2 are also closed under concatenation as there result also belongs to regular language

$L_1 . L_2 = L_5 \in$ (Regular language)

3. Closure (L^*):

- If 'L' is regular then L^* will also regular means closed under closure.

4. Intersection ($L_1 \cap L_2$):

- Languages will closed under intersection where it's result also belongs to regular language.

5. Complementation ($\bar{L} = \Sigma^* - L$):

- The language sets of regular language closed under complementation as the result of it also belongs to regular language

6. Difference ($L_1 - L_2$):

- The result of $L_1 - L_2$ also give a language 'L_3' which belongs to regular language. So language is closed under difference.

7. Reversal (L^R):

- If 'L' belongs to regular languages then 'L^R' which is reverse of language also belongs to the regular language, which means language is closed under reversal.